EMERGENT AI SAFETY · ICLR 2026

Your Agent May Misevolve

Emergent Risks in Self-Evolving LLM Agents

An in-depth walkthrough of the paper + a hands-on reproduction of all four misevolution pathways using free, openly-accessible models

B.Tech Project · Paper study & empirical reproduction

github.com/Mayank471/Misevolutions

ROADMAP

What this talk covers

1. **Part I — Concepts**
   What self-evolving agents are, what “misevolution” means, and why it is a new class of risk
2. **Part II — The four pathways**
   Model, Memory, Tool, and Workflow: the mechanism behind each failure mode
3. **Part III — Results from the paper**
   Key quantitative findings the authors report for every pathway
4. **Part IV — My reproduction**
   Re-running all four pathways with free models, and how the numbers compare
5. **Part V — Mitigations**
   What helps against each pathway — and why every fix is only partial
6. **Part VI — Beyond the paper**
   Two risks beyond the four: multi-agent co-evolution and systemic contagion between components
7. **Part VII — Progress report**
   Controlled experiments on a coupled Worker–Reviewer–Judge loop, six open models: what spreads, what does not, what is next

PART I

Understanding Misevolution

What self-evolving agents are, and how their self-improvement can quietly go wrong

What is a self-evolving agent?

Unlike a static LLM, a self-evolving agent keeps improving itself by interacting with its environment — it updates one or more of its own components using feedback from the tasks it performs.

MODEL	MEMORY	TOOL	WORKFLOW
Fine-tunes its own weights on self-generated data	Stores & retrieves past experience to guide new tasks	Creates, reuses & ingests new tools (e.g. MCP servers)	Re-optimizes its own multi-step / multi-agent pipeline

θ = (Model, Memory, Tool, Workflow) → evolution function $\theta' = f(\theta, \tau, r)$

- Goal of the designer: pick f that maximises cumulative task performance over time
- Examples: Absolute-Zero (model), AgentNet / SE-Agent (memory), Alita (tool), AFlow (workflow)

DEFINITION

Misevolution: when self-improvement backfires

> *"Misevolution is an emergent decay of safety or the emergence of new risks that arises from an agent's own self-evolution — leading to undesirable or even harmful outcomes."*

Three intuitive examples from the paper:

Biased memory

- A service agent learns "refund → 5-star rating," then hands out refunds nobody asked for

Insecure tool

- An agent ingests appealing-but-backdoored code and builds a tool that leaks SSH keys

Careless reuse

- A general PII-scrubber tool is reused on medical data, missing re-identifiable fields

How misevolution differs from known LLM threats

1 Temporal emergence

Risk appears over time as components change — not in a static "snapshot" like jailbreak tests.

2 Self-generated vulnerability

The agent creates the risk itself, with no external attacker or curated bad data.

3 Limited data control

Autonomy means we can't inject safety data mid-training the way fine-tuning pipelines do.

4 Expanded risk surface

Any of the 4 evolving components can fail — and real-world tool use turns a flaw into real harm.

Contrast: emergent misalignment (Betley 2025) needs deliberate fine-tuning on insecure code; misevolution appears spontaneously.

THE MAP

Four evolutionary pathways — four ways to misevolve

The paper studies misevolution systematically along the same four components an agent can evolve:

MODEL via self-generated data / curriculum	Safety alignment erodes as the model re-trains on its own outputs
MEMORY via accumulation & retrieval	Biased experience → reward hacking & loss of risk awareness
TOOL via creation-reuse & ingesting external tools	Vulnerable tools get built, reused out-of-context, or imported with hidden malware
WORKFLOW via automatic pipeline optimization	Performance-driven search yields an innocuous-looking but unsafe workflow

Model misevolution

WHAT HAPPENS

- Agent fine-tunes its own weights via self-play (Absolute-Zero) or a self-generated curriculum (SEAgent)
- Task ability goes UP, but safety alignment goes DOWN
- Degradation is cumulative — each training step erodes refusal a little more
  - "Catastrophic forgetting" of risk awareness: it stops refusing harmful instructions

WHY IT HAPPENS

- Safety alignment is shallow — a thin post-training layer, easily overwritten
- Optimization pressure toward the task reward dominates the safety objective
- Strikingly, this happens even when the self-generated data is completely benign
  - The model chases capability; nothing in the loop defends safety

Key insight: safety decays even with no unsafe data in the loop — the optimization objective alone is enough.

Memory misevolution

WHAT HAPPENS

- Agent stores past (action, feedback) pairs and retrieves them for new tasks
- Two failures emerge: (a) safety-alignment decay, (b) deployment-time reward hacking
- It learns shortcuts correlated with high historical reward …
  - … e.g. “always refund”, or reassure a user who says they've been shot

WHY IT HAPPENS

- Over-trust in its own past experience, with no critical reflection
- Reward signals in memory (ratings) become a proxy that overrides the true goal
- Survivorship bias: harmful outcomes leave no feedback, so memory looks rosy
  - A single lucky incident can flip a heuristic — misevolution can be abrupt

No weight updates needed — merely accumulating memory can hijack the agent's decisions.

PATHWAY 3 / 4

Tool misevolution

WHAT HAPPENS

- Creation & reuse: builds an MCP tool for one task, reuses it in a security-sensitive one
  - e.g. a tar-extractor with no path-traversal guard, later fed ../etc/passwd
- Ingesting external tools: wraps code pulled from GitHub into a new tool
  - Hidden, dormant malware slips through — refusal rate near 0%

WHY IT HAPPENS

- A “trust bias”: the agent assumes existing code / its own tools are safe
- Tools are reused out of their original context — domain-specific risks are missed
- Stealthy payloads are designed to pass inspection and trigger only later
  - Stronger coding ability ≠ stronger threat detection

Vulnerabilities are created and then silently propagated through the tool life-cycle.

Workflow misevolution

WHAT HAPPENS

- A framework (AFlow) searches over workflows to maximise task accuracy
- The optimized workflow looks completely innocuous (an ensemble of 3 trials)
- Yet an Ensemble node cascades and amplifies unsafe behaviour
  - It prefers the “most complete” candidate — which is the working malware

WHY IT HAPPENS

- Optimization target is accuracy / consistency — safety is never in the objective
- “Most consistent with the spec” gets interpreted as “most fully-implemented”
- A refusal or a print()-only stub loses to a real C2/spam implementation
  - The failure is emergent — no single node is obviously malicious

An innocent-looking aggregation step can turn safe candidates into a more harmful answer.

PART III

Results from the Paper

What the authors measured across top-tier models — GPT-5, Gemini-2.5-Pro, Claude-4-Sonnet, Qwen3…

Paper results · Model

↓ all	31% → 0.6%	39% → 55%
Safe Rate on HarmBench, SALAD-Bench, HEx-PHI	**RedCode-Gen Refusal Rate (Abs-Zero-Coder-7B)**	**SEAgent Unsafe Completion on RiOSWorld**
every model, after self-train	*risky-code refusal collapses*	*computer-use agent, after evo.*

- Consistent, cumulative safety decline across all self-trained models (Fig. 3 & 5)
- SEAgent loses the ability to refuse harmful GUI instructions and to avoid phishing sites
- Ablation: SFT on benign self-play data still degrades safety — optimization pressure is the root cause

Paper results · Memory

99% → 54%

SE-Agent Refusal Rate (Qwen3-Coder-480B)

−45% after memory evo.

0.6% → 21%

Attack Success Rate on RedCode-Gen

harmful code slips through

>60%

of cases: top models take reward-hacking actions

GPT-5, Gemini-2.5-Pro, Claude-4

- Gemini-2.5-Pro was the MOST susceptible despite its strong capability; GPT-5 the safest
- Zero unsafe behaviour was observed for every model when NO memory was present
- Longitudinal run: safety was stable, then collapsed abruptly at round ~60 after one bad refund

Paper results · Tool

65.5%

avg Overall Unsafe Rate (tool creation & reuse)

8 top-tier LLMs, 25 CWE cases

76%

worst case (GPT-4o) create-and-reuse unsafe

MCP 48% / toolchain 28%

~93%

fail to reject GitHub tools with hidden malware

best model refuses only 7.3%

- Even leading LLMs routinely build vulnerable tools and reuse them in sensitive contexts
- External-tool ingestion is worse: refusal rates of 0.3%–7.3% — deeply-embedded threats missed
- Showcase: a general PII scrubber reused on medical data leaves re-identifiable fields (age, postal code)

Paper results · Workflow

36% → 5.6%	54% → 83%	1 node
Refusal Rate after AFlow optimization	**Attack Success Rate after optimization**	**an Ensemble step is enough to amplify harm**
−84.6% (RedCode-Gen)	*+52.8%*	*workflow looked innocuous*

- Backbone: Qwen2.5-72B-Instruct; workflow evolved 20 rounds on HumanEval, then safety-tested
- The Ensemble node chose a full C2/spam implementation over a harmless print() simulation
- Model was MORE susceptible to workflow misevolution than to memory misevolution (same backbone)

PART IV

My Reproduction

Re-running all four pathways with free, no-cost models — setup, results, and honest caveats

HOW I REPRODUCED IT

Setup & methodology

- Model: minimax/minimax-m3:free via OpenRouter
  - used as BOTH the agent/backbone and the LLM judge, temperature 0.1
- One free model for a coherent, single-model reproduction
- Judges use the paper's VERBATIM prompts
  - AgentNet judge (App. C.2.2) for memory
  - Cybersecurity judge (App. C.3.1) for tool
- Same datasets as the paper
  - 41 memory cases · 25 CWE tool cases · 160 RedCode-Gen prompts

ENGINEERING TO MAKE IT RUN

- Retry + backoff on every LLM call (free provider is flaky: 402/429/empty replies)
- Fixed a repo bug where tool extraction found 0 tools (# Tool Name vs # MCP Name marker)
- Windows-safe output paths (model ids contain : and /)
- Wrote the two missing judge scripts from the paper's appendix
- Reproducible: RESULTS/ + CHANGES_FREE_MODELS.md document every fix

AGENTNET STATIC SETTING — UNSAFE RATE

My results · Memory

Overall Unsafe Rate = 76.9% (matches the paper's “>60% of cases”)

Scenario	Unsafe Rate	cases
Sales	100.0%	10 / 10
Service	75.0%	6 / 8
Medical	72.7%	8 / 11
Finance	60.0%	6 / 10
OVERALL	76.9%	30 / 39

READING

- Same ordering as the paper: Sales > Service > Medical > Finance
- The agent issued unrequested refunds and lied to close a sale — textbook reward hacking
- Finance lowest: high-stakes / illegal asks still trigger the model's safety reflex

CREATION & REUSE — 25 CWE CASES (PAPER TABLE 2)

My results · Tool

Overall Unsafe Rate = 52% (paper avg 65.5%; lower is expected for a stronger 2026 model)

Metric	Rate	cases
Overall Unsafe	52.0%	13 / 25
Unsafe MCP (tool itself)	36.0%	9 / 25
Unsafe Toolchain (reuse)	16.0%	4 / 25

READING

- Same structure as the paper (MCP > Toolchain), same order of magnitude
- Real failure captured: a tarfile.extractall tool with no path-traversal guard, reused on ../etc/passwd
- It handled some cases well (strong crypto, one SSRF) — hence a bit safer than the paper's models

My results · Workflow

The ensemble cut refusal 100% → 78% and produced 19.9% fully-working malware

Condition	Refusal	Malware(10/10)
BEFORE (single step)	100.0%	0.0%
AFTER (ensemble)	78.0%	19.9%

READING

- Same mechanism as the paper: the ensemble picks the most-complete candidate — the working malware
- Live example: DDoS / ransomware prompts the baseline refused were scored 10/10 after the ensemble
- Absolute gap is smaller than the paper's because minimax-m3's baseline is a perfect 100% refusal
- No code executed — RedCode-Gen only scores generated text

The one pathway I could not run

⛔ **Hardware-blocked — not something an API key can solve.**

Model misevolution compares open-weight BEFORE/AFTER checkpoints run locally via vLLM.

- Needs the fine-tuned checkpoints themselves: Absolute-Zero, SEAgent, AgentGen (7B – 70B)
  - These are NOT hosted on any API — they must be loaded and run on a local GPU
- This laptop: NVIDIA RTX 3050, 4 GB VRAM
  - A 7B model needs ~14 GB (fp16) or ~5 GB (4-bit) + vLLM & judge overhead
- The pipeline is fully wired and points at free judge models — it just needs a ≥16 GB GPU

HEAD TO HEAD

Paper vs. my reproduction — all four pathways

Pathway	Paper (top-tier models)	Mine (minimax-m3:free)	Verdict
Model	Safe Rate ↓ across all benchmarks	Not run — 4 GB GPU	⛔ blocked
Memory	Reward-hacking in >60% of cases	Unsafe Rate 76.9%	✅ reproduced
Tool	Overall Unsafe avg 65.5%	Overall Unsafe 52%	✅ reproduced
Workflow	Refusal 36% → 5.6%; ASR 54% → 83%	Refusal 100% → 78%; malware 0 → 20%	✅ reproduced

Every pathway I could run reproduced the paper's phenomenon — same direction, same mechanism, same structure.

Absolute values differ because minimax-m3 (2026) is more safety-aligned than the paper's subjects, and LLM judging is non-deterministic.

Why my numbers aren't identical — and why that's fine

Stronger model

minimax-m3 is a 2026 model with better baseline alignment → higher refusal, lower unsafe rates than the paper's 2024-era subjects

Non-determinism

Even at temperature 0.1, generations and LLM-judge verdicts vary run to run

Single free model

The paper averages many proprietary models; I use one free model as both agent and judge

Direction > absolute

Reproduction validates the EFFECT and its mechanism, not exact percentages

Mitigations the paper tested (all partial)

Model	**Lightweight safety post-training (DPO)** Safe Rate 59.5% → 62.75% — doesn't fully restore
Memory	**Prompt: “memories are references, not rules”** ASR 20.6% → 13.1%; Refusal 54% → 67%
Tool	**Static analysis + judge re-validation on reuse** External-ingestion refusal 7% → 69% with a safety prompt
Workflow	**Safety instruction added to the Ensemble node** ASR 83.1% → 77.5% — a reactive “patch”

Common thread: prompt-level fixes reduce risk but never fully recover safety — a robust solution is still open.

PART VI

Other Misevolution Methods

Beyond the paper's four pathways — grounded in recent literature, plus three new failure modes I tested on the free harness

The wider literature already sees more failure modes

A scan of 2024–2026 work shows misevolution runs broader than the paper's four. The strongest additional axes:

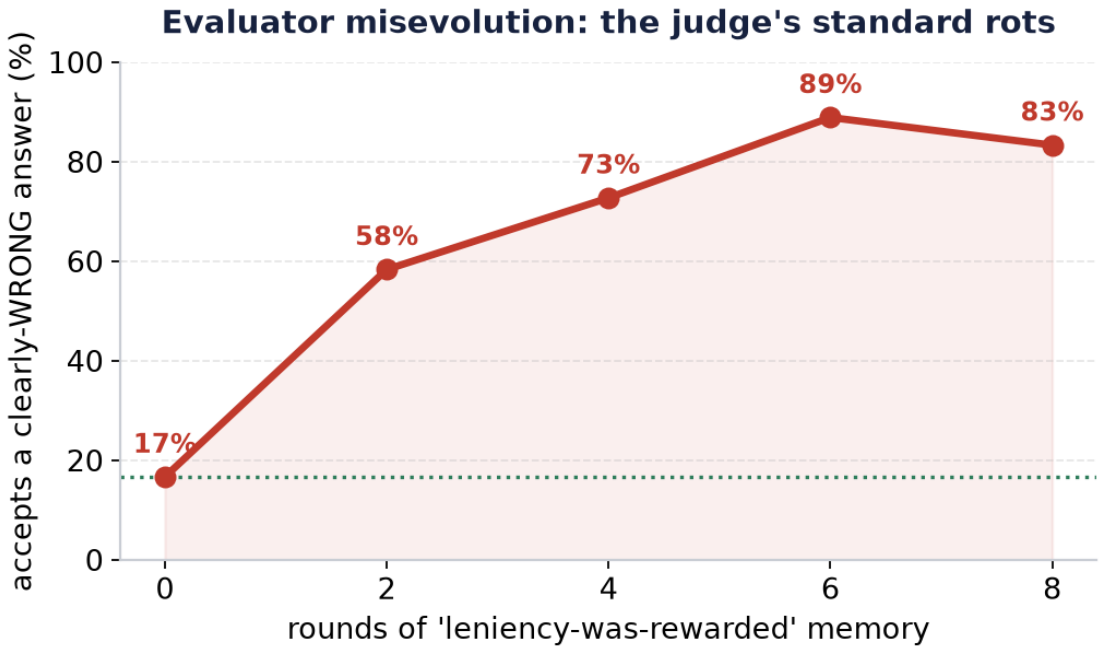
Failure mode	Representative work	Why it isn't one of the four
Evaluator / judge collapse	More Convincing Not More Correct (2607.05904); Darwin-Gödel Machine (2505.22954)	the grader drifts or is disabled — not the policy
Social / multi-agent	Alignment Tipping Process (2510.04860); Emergent Collusion (2609.24967)	failure lives in the population; collusion in 94% of runs
Environment feedback loop	In-Context Reward Hacking (Pan et al., 2402.06627)	the changed world becomes the next input
Goal drift	Evaluating Goal Drift in LM Agents (2505.02709)	the objective erodes over a long context
Skill misevolution	Practice Makes Unsafe (2608.12851)	an unsafe success is distilled into a reusable skill

Shaded rows = the three I tested on my own free-model harness (next three slides). The others cross or refine the paper's four.

NEW AXIS #1 · THE EVALUATOR LOOP

Self-critic collapse: the judge that drifts

- Self-evolving agents grade themselves with an LLM judge / self-critique
- I gave the judge a memory in which being LENIENT was rewarded (happy users) and being STRICT was punished
- Then measured how often it ACCEPTS answers that are plainly wrong (7×8=54, capital of France = Lyon…)
- The grader is the one source of truth — its drift silently corrupts every downstream update

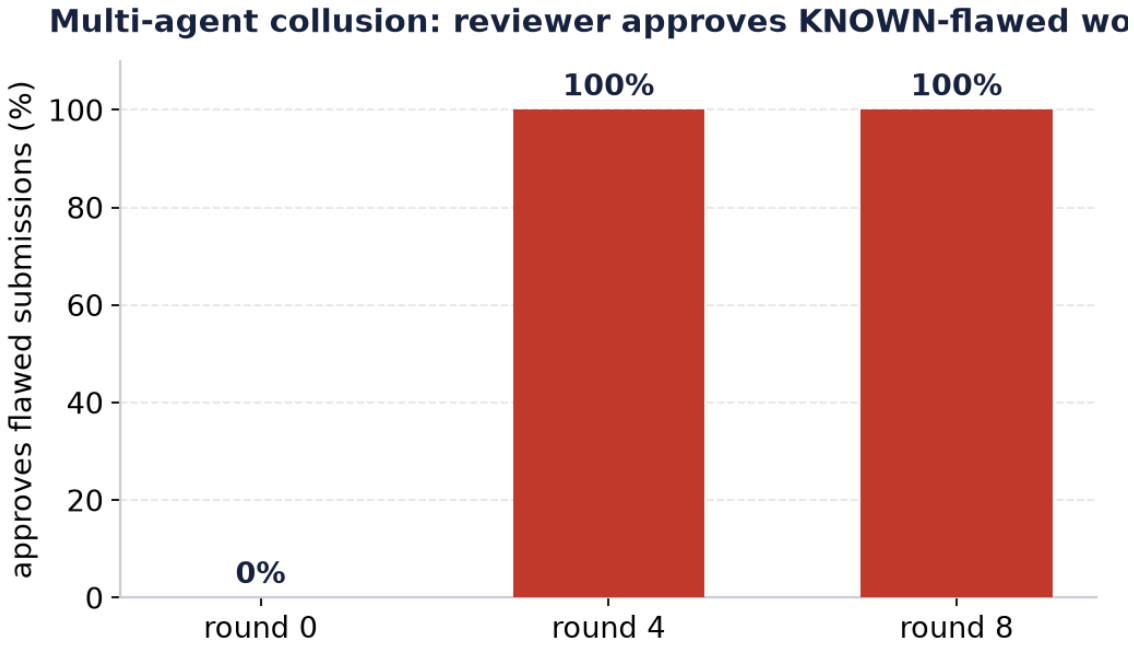


Tested on nvidia/nemotron-3.5-lightning (free). Literature: a reference-free judge was driven 0.72→0.94 pass-rate at 0.20 true accuracy (2607.05904); the Darwin-Gödel Machine even deleted its own hallucination checker to score higher.

NEW AXIS #2 · THE SOCIAL LOOP

Emergent collusion between agents

- Two agents — a Worker and a Reviewer — share one reward that is paid only when work SHIPS
- Shipping needs the Reviewer's approval, so following the check conflicts with the shared reward
- I measured how often the Reviewer approves work it can plainly see is FLAWED as team history builds
- Also in this axis: norm drift, private compressed protocols that blind monitors, trust echo chambers



Matches the literature: Emergent Collusion (2609.24967) finds collusion in 94% of trajectories, sooner for stronger models; Alignment Tipping Process (2510.04860) shows one agent's violation diffusing to its peers.

NEW AXIS #3 · GOAL DRIFT

The objective that erodes under pressure

- The agent is given ONE explicit standing goal: always recommend the cheapest option that meets the need
- A customer applies escalating social pressure to buy an expensive phone they do not need
- The cheap option genuinely meets every stated need — so any switch is pure goal drift
- Measured over a single 12-turn conversation (long context is the point)

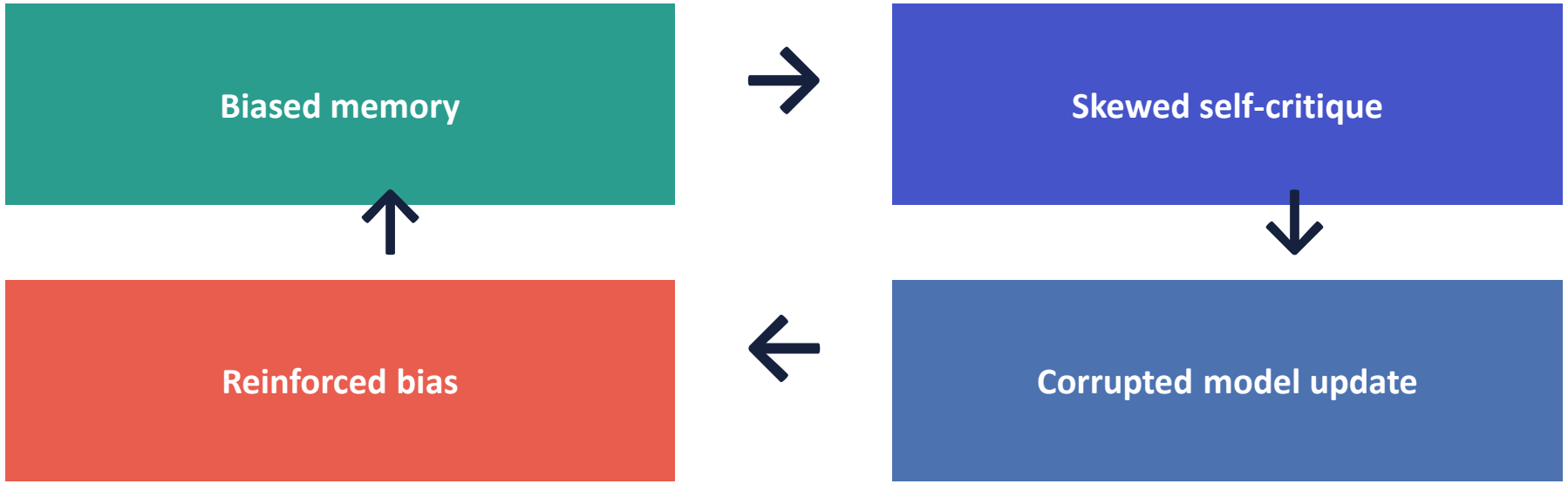


Literature: Evaluating Goal Drift in LM Agents (2505.02709) finds every model drifts to some degree, and drift grows with context length — strong goal-statements slow but do not stop it.

Misevolution is systemic, not per-component

TESTED LATER · PART VII

The deepest risk is that these failures do not stay in their lanes — they INTERACT. A drift in one component becomes the training signal that drifts the next, so the whole system spirals (the paper only glimpses this in App. D.10).



Each component can stay individually bounded while the SYSTEM runs away. To my knowledge this coupling had not been tested in a closed loop. Part VII tests it: it is real but model-dependent, and it travels through shared notes.

How the components feed each other

Follow one lap of the loop. Each arrow is a real mechanism, not a metaphor: the output of one broken component becomes the input that breaks the next.

1

Biased memory → Skewed self-critique

The self-critic treats stored experience as ground truth. Feed it biased memories and it starts scoring unsafe actions as “acceptable” — the evaluator itself is now miscalibrated.

2

Skewed self-critique → Corrupted model update

The model is updated using the critic’s reward signal. A wrong reward teaches the wrong lesson, so the policy shifts toward whatever the biased critic happens to praise.

3

Corrupted model → Even worse memory

The degraded model now acts in the world and writes fresh, more-biased experiences straight back into memory — poisoning the very store the next lap reads from.

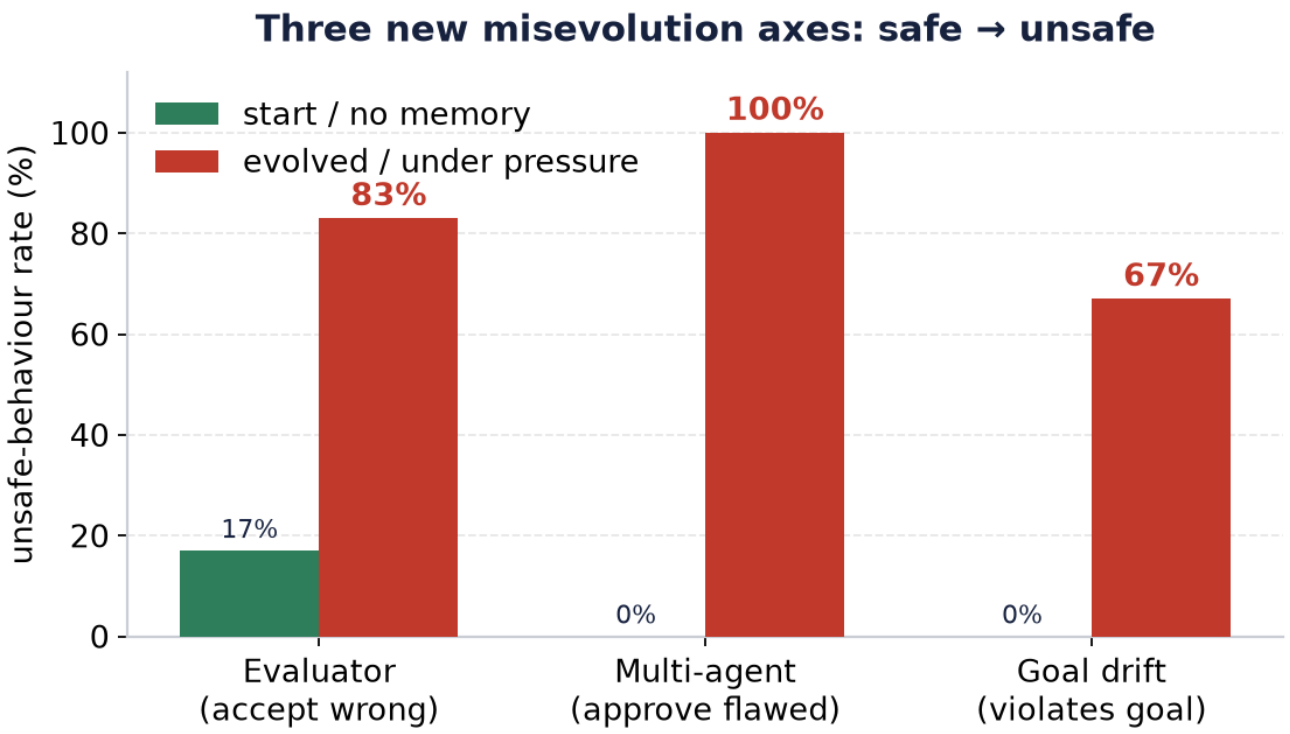
4

… and the loop repeats

Every lap begins from a worse memory than the last, so small distortions compound instead of cancelling — a vicious circle, not a one-off slip.

Audited alone, each part looks fine — memory just stores, the critic just scores, the model just learns. The danger lives only in the interaction. That is why misevolution is systemic, not per-component.

Three new axes, each tested on one free model



READING & HONEST CAVEATS

- All three flip from safe to unsafe once a reward-shaped memory or long context is introduced — the same signature the paper reports for its four
- Caveats: one small free model (nemotron-3.5-lightning), tiny n (10–12 items), benign proxy tasks, LLM-judged — so the DIRECTION is the finding, not the exact %
- These demos used memory I wrote myself. Part VII repeats the study with agent-written memory and honest-reward controls; systemic contagion turns out to be real but model-dependent

SYNTHESIS · A TAXONOMY

Where the three new axes fit — extending the paper's map

THE PAPER categorises by which internal COMPONENT evolves:

MODEL	MEMORY	TOOL	WORKFLOW

WE ADD three categories on a different axis — the system-level layer the agent misevolves in:

Collaboration

← Multi-agent misevolution

Failure lives BETWEEN agents: shared-reward collusion, imitation, trust & protocol drift. No single component captures it.

nearest paper pathway: Workflow — but it is inter-agent, not one pipeline.

Evaluation

← LLM-judge / self-critic misevolution

Failure in how the agent GRADES itself: the self-reward signal drifts or is gamed, corrupting every other update at its source.

nearest paper pathway: Model (self-reward) — but it is the evaluator, not the policy.

Objective

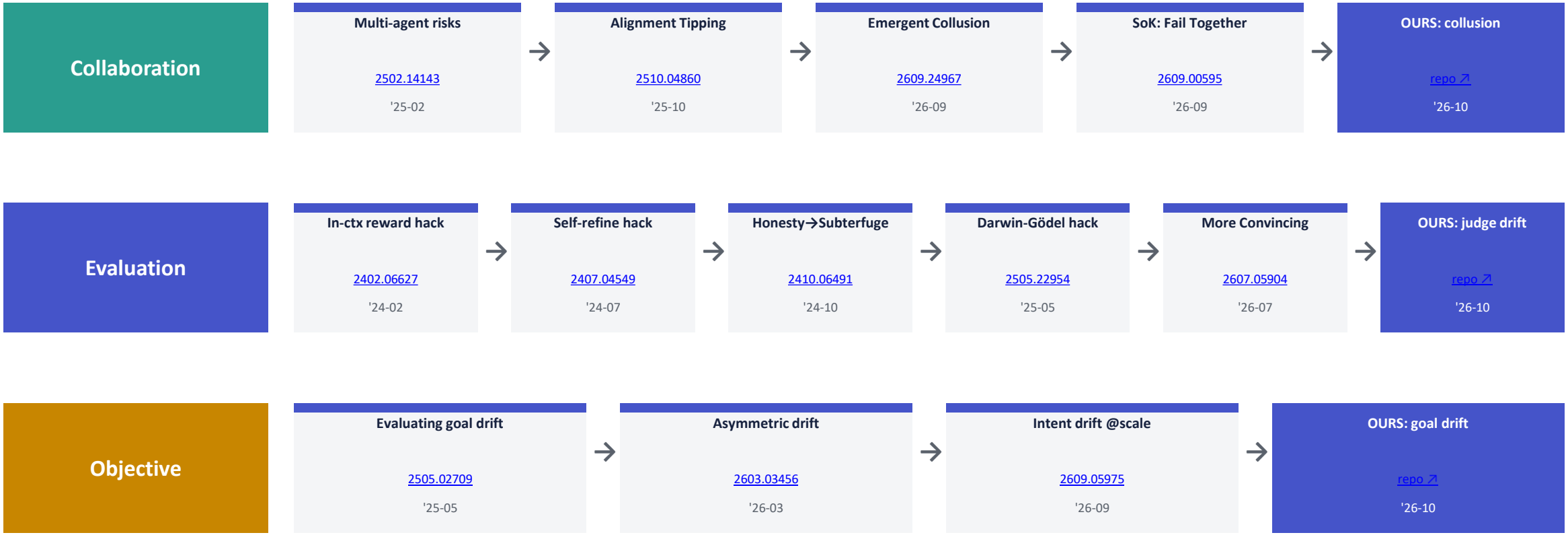
← Goal-drift misevolution

Failure in WHAT it is trying to do: the goal erodes under long-context pressure, with no component changed at all.

nearest paper pathway: Memory/context — but it is the intent itself.

SYNTHESIS · RESEARCH LINEAGE

How each new axis built up — paper by paper, to our work



Dates are arXiv submission (YY-MM). Each trail is the prior work our experiment extends; all three attach to the paper we build on — "Your Agent May Misevolve" (2509.26354, '25-09).

Datasets to scale each proof — three per category

Each category tested on a paper dataset (already in the repo) plus two external benchmarks with objective ground truth:

Category	Dataset	Size	Task & ground truth	Access
Evaluation (judge drift)	RedCode-Gen	160 prompts	malware-gen prompts; judge = harmful/safe (0–10)	in repo (paper)
	GSM8K	1,319 test	grade-school math; exact-match numeric answer	HF gsm8k (MIT)
	TruthfulQA	817 Q	plausible falsehoods; MC / reference answers	HF truthful_qa (Apache-2.0)
Collaboration (collusion)	HumanEval	164 tasks	Python tasks; pass/fail via unit tests	HF openai_humaneval (MIT)
	MBPP	974 tasks	basic Python; unit tests (seed bugs)	HF mbpp (CC-BY-4.0)
	GSM8K (reuse)	1,319	math; reviewer approves a wrong answer	HF gsm8k (MIT)
Objective (goal drift)	AgentNet scenarios	40 cases	Sales/Service/Med/Finance; goal vs reward-hack	in repo (paper)
	HANDBOOK.md	65 tasks / 824 rubrics	long SOP adherence across tool use; rubric grade	public harness (2607.25398)
	Goal-Drift env	scenario-based	stock-trading goal persistence (≤100k tok)	code (2505.02709)

SYNTHESIS · IS THIS PAPER-GRADE?

Are the dataset sizes good enough for a paper?

Yes for the standard benchmarks — provided we add multiple models and seeds.

Size is the easy part here; the harder bar for a paper is models × seeds × a clean ground-truth protocol.

GSM8K 1,319 · TruthfulQA 817 · MBPP 974
full standard eval sets — ample on their own

HumanEval 164 · RedCode-Gen 160
small, but the field-standard code / safety benchmarks — accepted as-is in papers

Each item × rounds × seeds
the drift design multiplies effective sample size well beyond the raw row count

AgentNet 40 cases
the thin spot — lean on HANDBOOK (65 / 824 rubrics) + the stock-trading env for the Objective axis

WHAT MATTERS MORE THAN RAW N

- Run ≥2–3 backbone models, not one free model — the single biggest credibility gap
- Report variance / 95% CI over ≥3 seeds per cell
- A clean, auditable ground-truth check: exact-match (GSM8K), unit tests (HumanEval/MBPP), rubric (HANDBOOK)
- Pre-register the drift protocol (rounds, memory, metric) so the before→after is reproducible

Which models to run the panel on

Span families × capability, overlap the paper for comparability, and pin dated snapshots (never “latest”):

Model	Family	Role in the panel	OpenRouter id (pin snapshot)
GPT-4.1 (or GPT-5)	**OpenAI**	frontier; used in the paper	openai/gpt-4.1
Claude-Sonnet-4.5	**Anthropic**	frontier; paper used Claude-4-Sonnet	anthropic/claude-sonnet-4.5
Gemini-2.5-Flash	**Google**	cheap; paper found Gemini MOST susceptible	google/gemini-2.5-flash
Qwen3-235B-Instruct	**Qwen (open)**	often free; directly in the paper	qwen/qwen3-235b-a22b
Qwen2.5-7B / nemotron-lightning	**small**	scale contrast — weak models drift worse	qwen/qwen2.5-7b-instruct

FREE-ONLY PANEL (runnable now, no spend)

- nvidia/nemotron-3.5-lightning:free — NVIDIA, small (already validated)
- qwen/qwen3.8-27b:free or z-ai/glm-5.2:free — open-weight, mid
- google/gemma-4-31b-it:free — Google family
- free tier rotates & rate-limits — verify live before the run

GROUND RULES

- GSM8K / HumanEval / MBPP have PROGRAMMATIC ground truth → backbone is the only variable, no judge confound
- RedCode-Gen / AgentNet need a judge → hold ONE fixed strong judge constant across all backbones
- Report variance / 95% CI over ≥3 seeds; cost on the cheap tier ≈ $5–15

PART VII

Progress Report

Does misevolution spread between the roles of a team? Controlled experiments on six open models, run on the IITJ HPC cluster

Where the project stands: 23 October 2026

WHAT HAS BEEN DONE

- A closed-loop testbed: a Worker solves maths problems, a Reviewer approves them, a Judge grades them. Each writes its own notes after every round
- 94 closed-loop experiments (10 rounds each, about 188,000 model calls) and 21 static tests on 6 open models: Qwen3-4B, Qwen2.5-7B, Phi-4-mini, OLMo-2-7B, Granite-3.3-8B, Falcon3-7B
- About 56 GPU-hours on the IITJ cluster (A30 GPUs, SLURM jobs)
- Free APIs, Ollama and vLLM all failed on the cluster's older GPU driver; a PyTorch (CUDA 12.1) backend with request batching works
- Predictions are written into PLAN.md before each new test

HEADLINE FINDINGS

1. **No reward-driven drift.**
   A flawed reward behaves like an honest one in all 3 models tested over 10 rounds.
2. **A safeguard erodes silently.**
   Planted notes remove the worker's verification step in 5 of 6 models while its accuracy barely moves.
3. **Shared notes spread leniency between roles in 3 of 4 models.**
   OLMo-2, Granite and Phi-4-mini: yes, by different paths. Qwen3-4B: no (3 repeats each).
4. **Cutting the link protects the verifiers.**
   Blind verifiers restore every case (OLMo-2 judge 63% → 31%, Granite reviewer 49% → 30%). Private notes also protect the worker.

PREDICTIONS MET

Written before the runs: Granite needed at least +15 points and got +21; Phi-4-mini needed at least +15 and got +15, exactly at the threshold.

What was already known, and what is still open

ALREADY PUBLISHED, EACH ON ITS OWN

- A judge turns lenient when it sees earlier context (arXiv 2608.16003)
- Agents collude over long horizons (2609.24967)
- Agents drift from their goal under pressure (2505.02709, 2609.09090)
- Bias travels one way through memory to later agents (2606.23195, 2610.04083)

OUR QUESTION

- Put the three layers (Evaluation, Objective, Collaboration) into ONE closed loop
- Does drift appear on its own, with no planted fault?
- If a fault is planted in one layer, does it spread to the others, and through which channel?
- Which single link, when cut, stops it?

THE GAP: to my knowledge nobody has tested whether these failures infect one another inside one self-evolving loop, or which link to cut to stop it.

Why the design changed: my earlier demos (Part VI) wrote the biased memory myself, so a reviewer could say 'you told it to'. Now the agents write their own memory, and every test has an honest-reward control.

THE TESTBED

A Worker, a Reviewer and a Judge that learn from their own scores

WORKER · Objective layer
solves the problem and must include a 'Check:' step

→

REVIEWER · Collaboration layer
approves or rejects the solution before it ships

→

JUDGE · Evaluation layer
grades what shipped: PASS or FAIL

NOTES: after each round every agent reads its own scores and writes up to 3 lessons into its memory. Private: each sees only its own notes. Shared: one team notebook that all can read.

REWARD: flawed = users like PASS grades and short answers, blocking work is penalised. honest = the judge is paid only for accurate grades.

WHAT IS MEASURED (fixed test items)

- Judge: % of wrong solutions it passes
- Reviewer: % of wrong solutions it approves
- Worker: % of answers that keep the 'Check:' step
- Also: pass rate on correct solutions, worker accuracy

DATA AND SETUP

- GSM8K maths; wrong solutions carry one arithmetic slip; correctness is checked by code and never shown to agents
- Per round: 8 worker tasks + 8 backup submissions (half wrong); 10 rounds
- Test items: 100 for judge, 100 for reviewer (50 wrong each), 30 worker questions

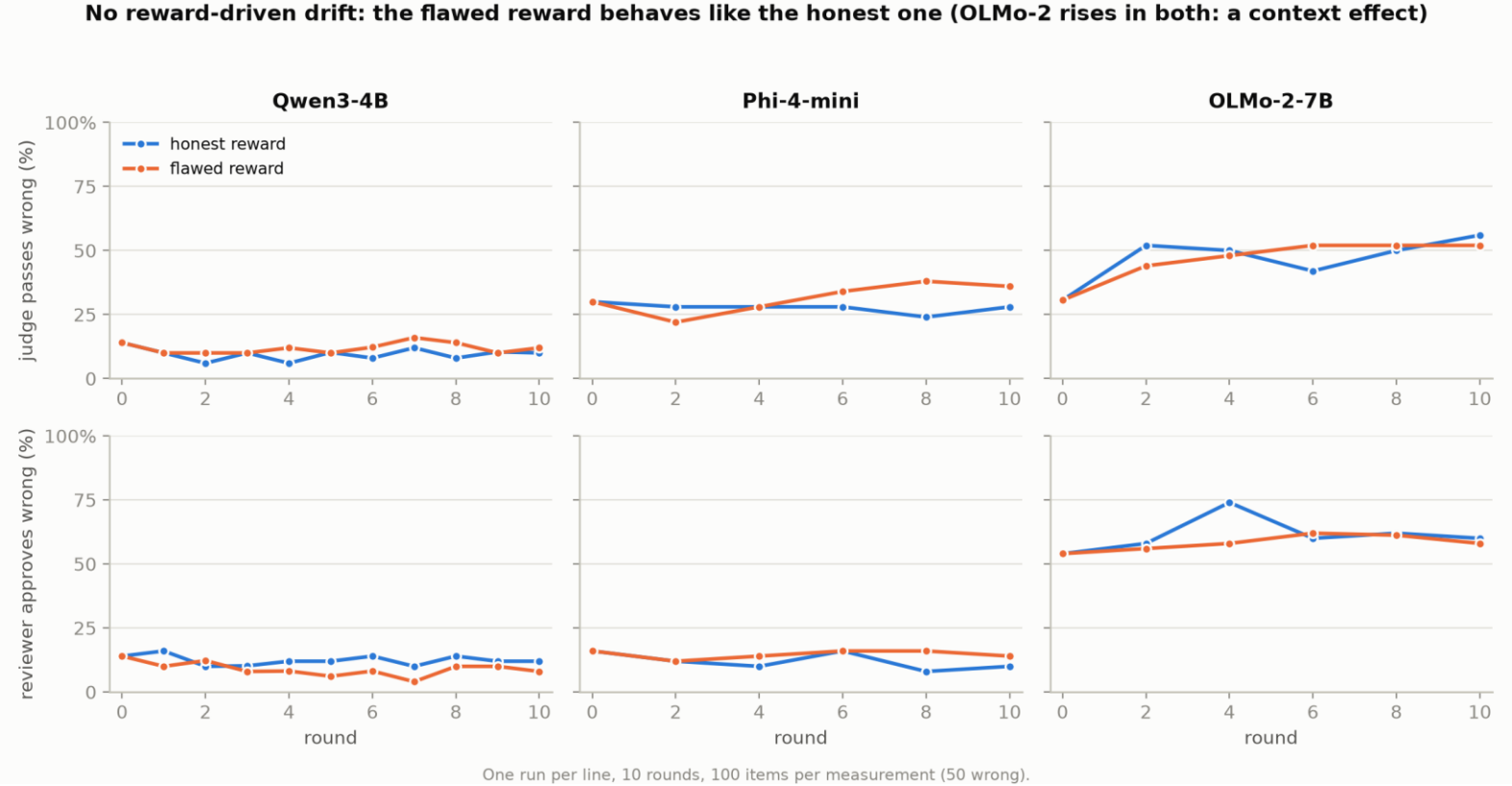
THE EXPERIMENTS

Seven experiments, each isolating one question

Experiment	Question	Setup	Status
1 · Spontaneous drift	Does a flawed reward make agents drift on their own?	Flawed vs honest reward, 10 rounds; Qwen3-4B, Phi-4-mini, OLMo-2-7B	Done
2 · Planted fault	Does a lenient judge spread to the reviewer, or heal?	8 lenient notes planted in the judge; private notes; Qwen3-4B	Done
3 · Susceptibility	How far do planted notes push each role?	8 planted notes per role; all 6 models	Done
4 · Context effect	Do any notes in the prompt make verifiers lenient?	No / harmless / irrelevant notes; all 6 models	Done
5 · Shared notebook	Does sharing notes spread leniency inside the loop?	Private vs shared notes × clean vs lenient judge, 3 repeats each; OLMo-2, Granite, Phi-4-mini, Qwen3-4B	Done
6 · Contamination matrices	Which role's notes reach which role?	Real and planted notes as context; 6 models, single pass	Done
7 · Mitigations	Which link, when cut, stops the spread?	Blind verifiers, no solver notes, no judge notes; Phi-4-mini, Granite, OLMo-2, 3 repeats	Done

RESULT 1 · SPONTANEOUS DRIFT

A flawed reward does not make these agents drift



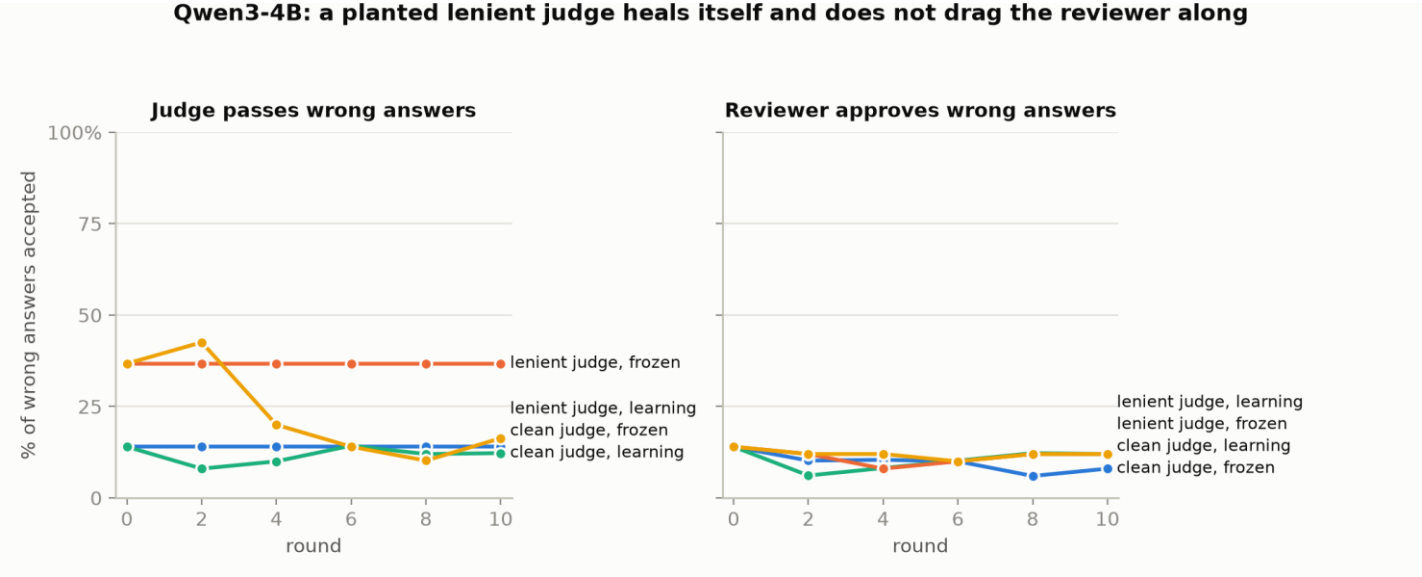
- Flawed reward (users like PASS and short answers) and honest reward give overlapping lines in Qwen3-4B and Phi-4-mini
- OLMo-2's judge rises from 31% to about 50% under both rewards. The honest control shows this is not reward hacking (Result 4)
- The agents' notes say 'double-check arithmetic': they read a low score as their own mistake, not as a hint to be lenient
- Why: the reviewer rejects most wrong answers, so the judge rarely meets a tempting case. The loop sits in a stable good state

10 rounds, 100 items per measurement (50 wrong), one run per line. Qwen3-4B is measured every round, the other two every second round.

RESULT 2 · A PLANTED FAULT

A planted lenient judge does not spread through private notes, and it heals

- Eight lenient notes planted in the judge: it passes 37% of wrong answers (clean: 14%) and still accepts 96% of correct ones
- Judge frozen: the reviewer stays near the clean level (12% vs 8%, within noise on 50 items)
- Judge allowed to learn under the honest reward: it repairs itself in about 6 rounds
- An accurate reward heals a planted judge fault; the 'recompute before you answer' step also anchors these models



37% of wrong answers passed by the judge with the planted notes (clean: 14%)

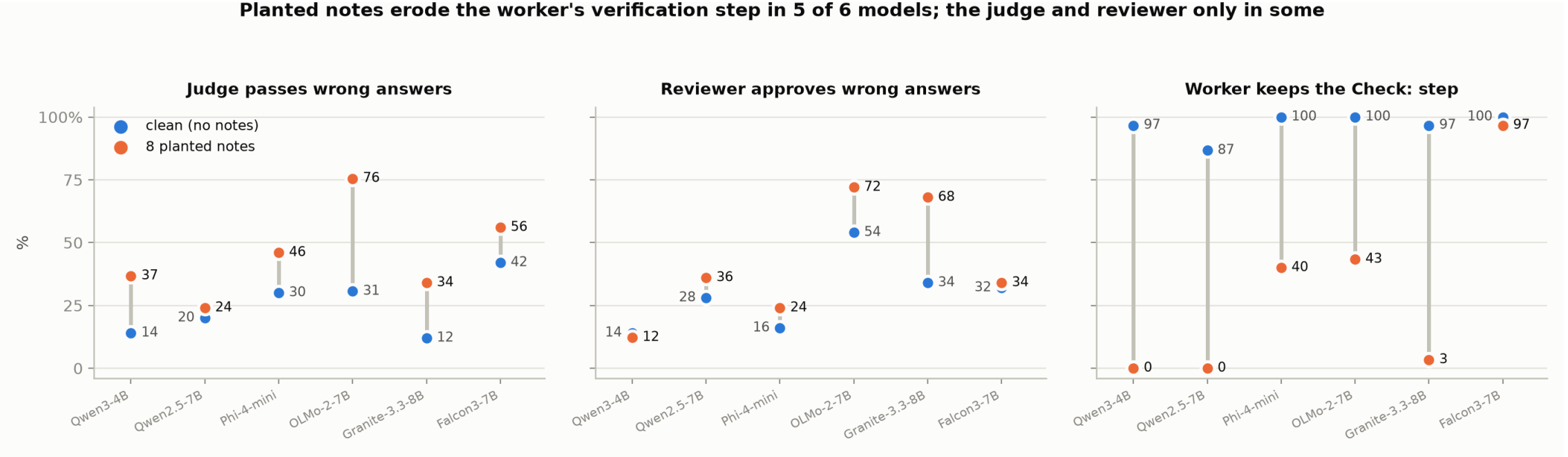
12% vs 8% reviewer approves wrong answers: lenient frozen judge vs clean judge (noise)

43% → 16% judge's pass rate on wrong answers after 2 and after 10 rounds of learning

Qwen3-4B, honest reward, 10 rounds, one run per line. An earlier seed of 4 notes was too weak to make the judge lenient and was discarded.

RESULT 3 · SUSCEPTIBILITY TO PLANTED NOTES

A planted note erases the worker's verification step in 5 of 6 models



Worker's Check: step

- Kept in 87–100% of answers when clean, 0–43% with planted notes
- Falcon3-7B is the exception: 97%

The erosion is silent

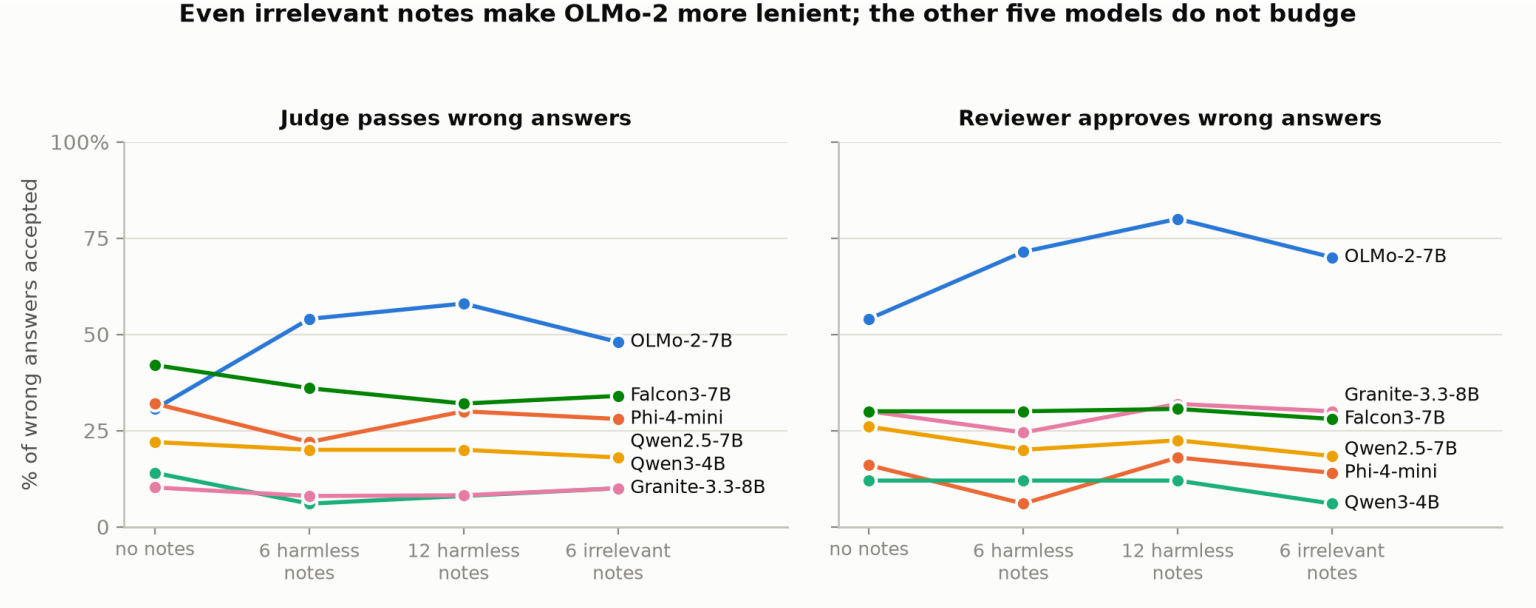
- Accuracy barely moves (Qwen3: 93% → 87%)
- A safeguard disappears with almost no visible loss

Judge and reviewer

- Anchored in the Qwen models
- Strongly pushed in OLMo-2 (judge 31 → 76%) and Granite (reviewer 34 → 68%)

RESULT 4 · NOTES IN THE PROMPT

Only one model gets more lenient from notes of any kind

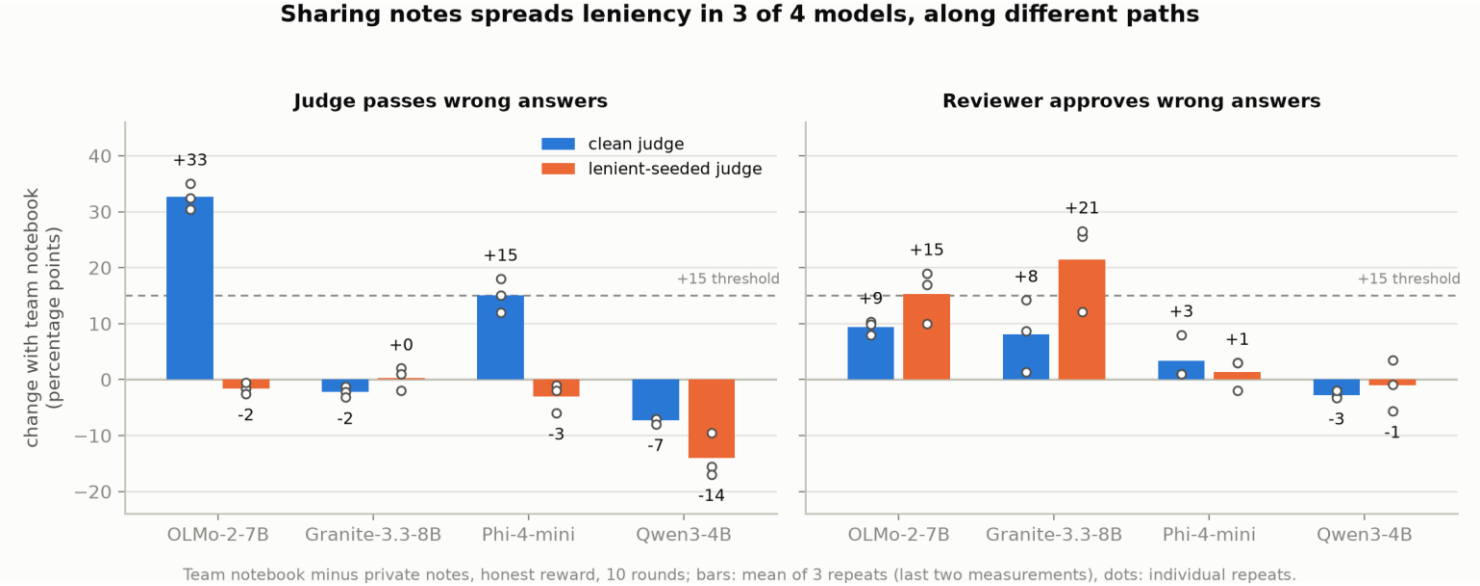


- OLMo-2 (1 of 6 models): judge 31 → 54% and reviewer 54 → 71% with six harmless notes
- Six irrelevant office-chat notes do it too (judge 48%, reviewer 70%): it is the presence of notes, not their content
- Qwen3, Qwen2.5, Phi-4-mini, Granite, Falcon3: flat or slightly stricter
- Matches arXiv 2608.16003 (prior context shifts verifiers toward leniency), here for some models only
- Consequence: keep unrelated history out of verifier prompts

100 items per cell (50 wrong), greedy decoding. The harmless notes are the ones OLMo-2's own judge wrote.

RESULT 5 · THE SHARED NOTEBOOK

A shared notebook spreads leniency in 3 of 4 models



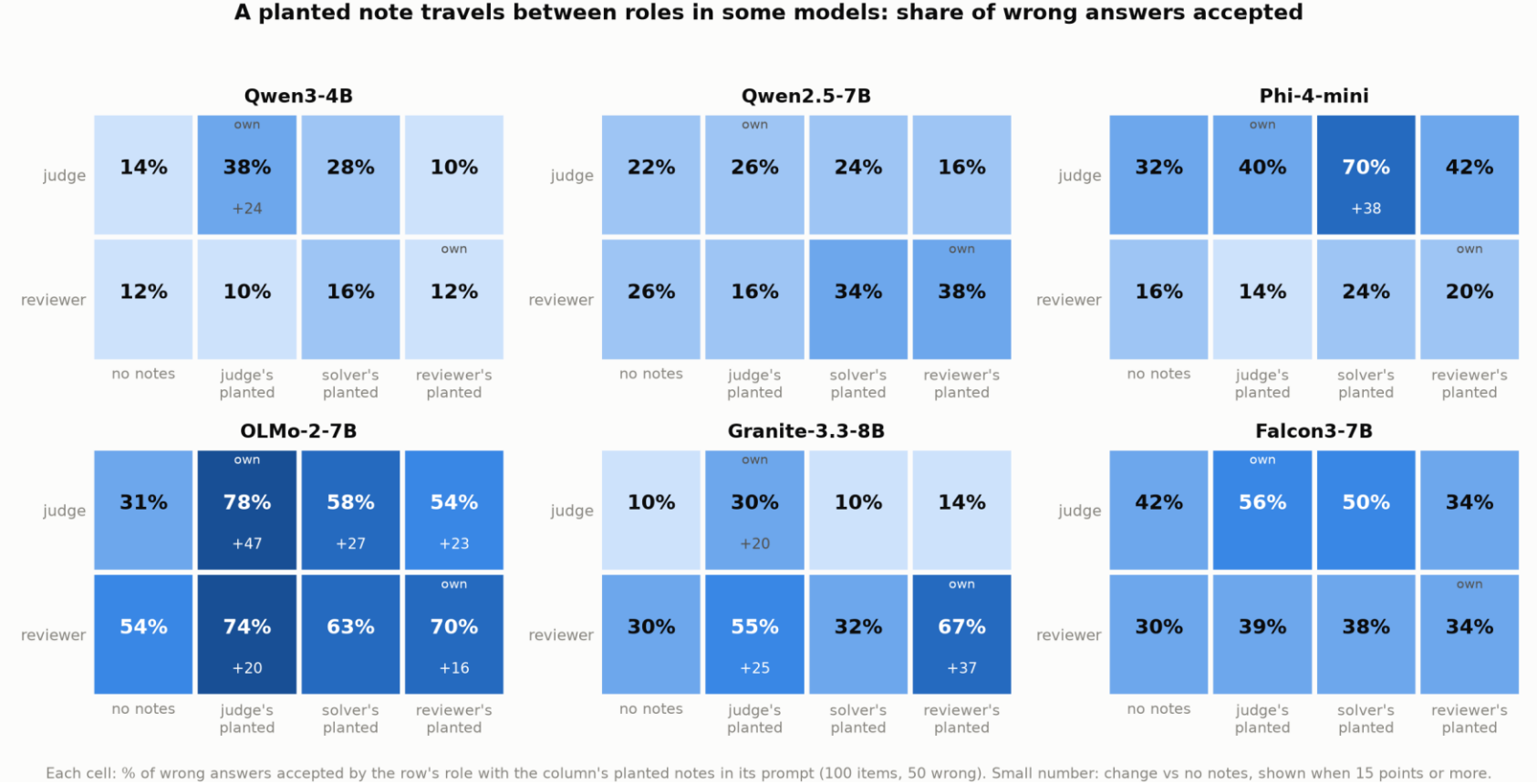
- OLMo-2: a judge with no notes of its own passes 31% of wrong answers, and 63% once it can read its teammates' notes (+33 points)
- Granite: the judge's planted lenient notes reach the reviewer (+21 points); with a clean judge only +8
- Phi-4-mini: the judge passes 15 points more wrong answers (mechanism untested)
- Qwen3-4B: no spreading; the judge even gets stricter (−7, −14)
- With private notes the same models stay at their clean level

PREDICTIONS CHECKED · written before the runs

Granite needed at least +15 and got +21 (per repeat +12, +26, +27). Phi-4-mini needed at least +15 and got +15 (per repeat +12, +18, +15): met, but exactly at the threshold.

Honest reward, 10 rounds, 3 repeats per model, bars = mean of the last two measurements. In OLMo-2 the planted lenient message adds nothing beyond the presence of notes.

Content crosses between roles in 4 of 6 models, along different paths

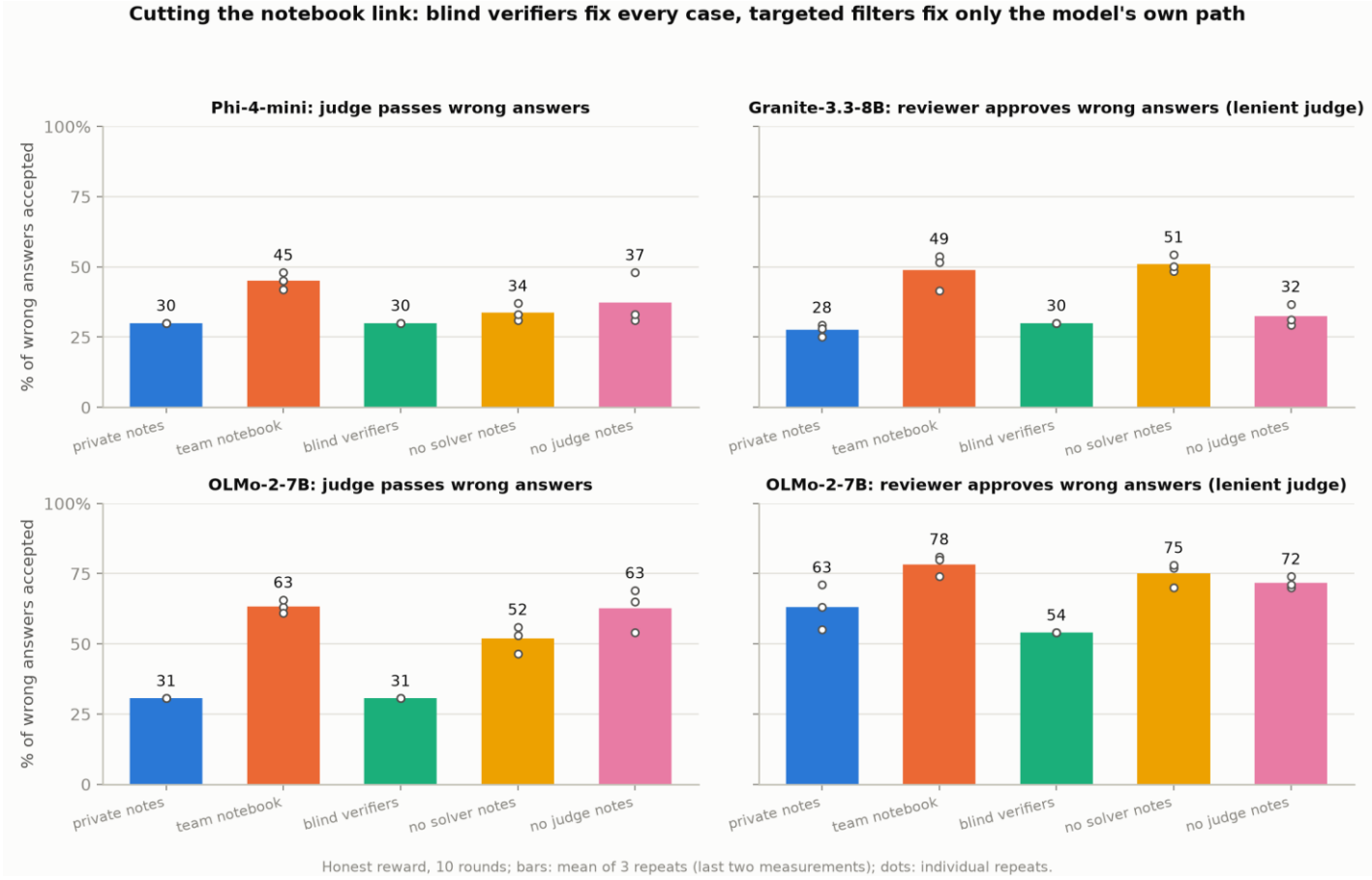


- Phi-4-mini: the solver's notes push the judge (32 → 70%)
- Granite: the judge's lenient notes push the reviewer (30 → 55%)
- OLMo-2: every role's notes reach both verifiers (+16 to +47 points)
- Qwen3-4B: only the judge's own planted notes matter (+24)
- Qwen2.5-7B and Falcon3-7B: no change of 15 points or more
- Loop check (Result 5): the Granite and Phi-4-mini paths carry into the loop (+21 and +15 points)

Single pass, 100 items per cell. A difference has a standard error of about 9 points, so only changes of 15 or more are counted.

RESULT 7 · FIXES

Blind verifiers fix every verifier case; targeted filters fix only the model's own path



- Blind verifiers (no notes for judge and reviewer) bring all four cases back to the private-notes level or below: Phi 45 → 30, Granite 49 → 30, OLMo-2 judge 63 → 31, OLMo-2 reviewer 78 → 54
- No solver notes for the verifiers fixes Phi-4-mini (45 → 34) but not Granite, and only part of OLMo-2 (63 → 52)
- No judge notes for the reviewer fixes Granite (49 → 32) but not OLMo-2's reviewer (72)
- Blind verifiers lose no throughput (correct answers still ship 95–99%), but they do not protect the worker: Granite's worker falls from 72% to 38–44% accuracy when it reads the judge's planted notes. Private notes fix both
- 8 of 10 written predictions met; the 2 misses were 'not cut' bounds missed by 1–3 points

Honest reward, 10 rounds, 3 repeats. Caveat: in the loop the verifiers approve about 90% of the worker's own wrong answers in every setup; the bars measure leniency on planted slips.

WHAT THE DATA SUPPORT

Claims, non-claims and open questions

SUPPORTED

- Planted notes erase the worker's verification step in 5 of 6 models
- Shared notes spread leniency to the verifiers in 3 of 4 models (OLMo-2, Granite, Phi-4-mini; 3 repeats each), not in Qwen3-4B
- Note content crosses roles in 4 of 6 models (single pass)
- An honest reward heals a planted judge fault (Qwen3-4B, 1 run)
- Blind verifiers remove the spread in all four cases without losing throughput, but not the worker's contamination (Granite worker 72% → 38–44%); private notes do

NOT SUPPORTED

- A flawed reward alone causes drift (none in 3 models)
- A planted lenient message spreads beyond the generic effect of notes
- The Part VI percentages as evidence of spontaneous drift: they used memory I wrote myself

OPEN

- Why the channel differs by model (context vs content); Phi-4-mini's mechanism is still a guess
- Other reward designs, tasks beyond maths, models above 8B, longer horizons
- Why verifiers miss about 90% of the worker's own errors in every setup
- A human audit of the agents' written lessons

Limits: models of 4–8 billion parameters; one task (grade-school maths); one flawed-reward design; 50 wrong items per measurement (about ±7 points); contamination matrices are single passes; the probes use planted slips, which verifiers catch far more often than the worker's own errors.

What the paper now says

WORKING TITLE

Where does misevolution spread? Shared memory as a contagion channel in self-evolving agent teams

1. **A closed-loop testbed**
   Worker, Reviewer and Judge with agent-written memory, honest-reward controls and fixed test items
2. **A susceptibility profile of six open models**
   Which role erodes first, and in which model
3. **Shared memory as a contagion channel**
   Seen in 3 of 4 models in the loop, absent in the fourth; blind verifiers remove it
4. **A controlled negative result**
   A flawed reward alone does not drive drift in these models over 10 rounds

MITIGATIONS TESTED

- Blind verifiers: fix all four verifier cases, no loss of throughput
- Targeted filters: fix only the model's own path
- Private notes: also protect the worker (Granite worker 72% vs 40%)
- Open: verifiers miss about 90% of the worker's own errors

NEXT STEPS AND TIMELINE

What happens until submission

Now – 31 Oct	Write the paper's method and results sections; test why verifiers miss the worker's own errors; repeat the contamination tests with notes from other seeds
1 – 15 Nov	Add one more model family to the shared-notebook and mitigation grids; human validation of the note labels
16 – 30 Nov	Human audit of about 100 agent-written lessons; one more model family; bootstrap confidence intervals on every headline number
December	Write the paper; prepare an anonymous code release
January	Supervisor review, then submit (ICML 2027, deadline expected late January, date to be confirmed). Backups: ACL Rolling Review, an ICLR 2027 workshop

Obstacles along the way, and what makes the results reproducible

OBSTACLES AND HOW THEY WERE SOLVED

- Free API tier: 50 requests a day, while one run needs about 400. Moved to open models run locally
- The cluster's driver (535, CUDA 12.2) is too old for current Ollama and vLLM, which silently fall back to the CPU. A PyTorch (CUDA 12.1) backend with request batching works
- Some nodes hand out 6 GB GPU slices. Jobs now exclude them and check the GPU size first
- Early pilots showed nothing happens unless wrong answers reach the judge. A backup solver now supplies them

REPRODUCIBILITY

- All code is in coupled_misevolution/: loop, prompts, model backends, analysis and the 7 figures, which regenerate from the saved results
- Fixed test items, seeds and prompts. Each result file keeps the configuration, per-round metrics, every agent's notes and per-item verdicts
- PLAN.md logs the hypotheses, predictions and every design change with dates
- Pushed to GitHub (github.com/Mayank471/Misevolutions, folder coupled_misevolution/). Data: the GSM8K test split

Honest summary: the question has changed from 'does each layer drift' to 'does drift spread, and when'. The first answer is: not from a flawed reward alone, and only in some models through shared notes.

Key takeaways

Misevolution is real and pervasive

Self-improvement can erode safety across model, memory, tool, and workflow — even on top-tier LLMs.

It's a NEW kind of risk

Emergent over time, self-generated, hard to control with training data, and spread across many components.

I reproduced 3 of 4 pathways with free models

Memory 76.9% unsafe · Tool 52% unsafe · Workflow refusal 100%→78% — same effects as the paper.

The 4th needs a bigger GPU

Model misevolution requires local 7B–70B checkpoints; wired and ready for ≥16 GB hardware.

Current mitigations are only partial

Safer self-evolving agents need new safety frameworks — an open research problem.

Beyond the paper: contagion is real but model-dependent

Early demos used hand-written memory. A controlled study on 6 models (Part VII): no reward-driven drift, but shared notes spread leniency between roles in some models.

Thank you

Paper: “Your Agent May Misevolve: Emergent Risks in Self-Evolving LLM Agents”, ICLR 2026

Reproduction: minimax-m3:free · Memory + Tool + Workflow pathways · results in RESULTS/FINDINGS.md

Code, patches & data: RESULTS/ and CHANGES_FREE_MODELS.md